Music Instrument rental & inventory system

A PROJECT REPORT

Submitted by

KRISHNAKUMAR SRIHARI

24150193

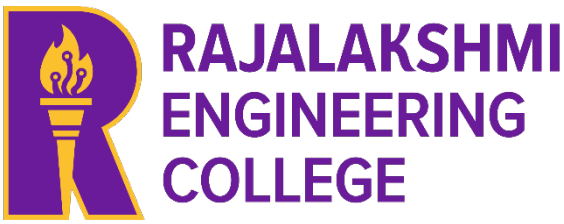
in partial fulfilment of the End Semester Examination for the course

CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF TECHNOLOGY
AIML



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

OCT 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled Music Instrument rental & inventory system is the bonafide work of Krishnakumar Srihari 241501093, who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year [2026–2027], Semester [V], under my supervision.

SIGNATURE	**SIGNATURE**
Dr. M.Ayyadurai HEAD OF THE DEPARTMENT Department of AIML Rajalakshmi Engineering College Thandalam – 602 105	Dr. Priya Thiyagarajan COURSE FACULTY Assistant Professor Department of AIML Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on

INTERNAL EXAMINER	**EXTERNAL EXAMINER**

DECLARATION

I, Krishnakumar Srihari 241501093, hereby declare that the project report titled “Music Instrument rental & inventory system ”, submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: [Chennai]
Date: 20/10/2026

SIGNATURE OF THE STUDENT
Krishnakumar Srihari
241501093

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman, Mr Meganathan M, Chairperson, Ms Thangam Meganathan and Vice-Chairman, Mr Abhay Meganathan for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal Dr Murugesan SN, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank the Head of the Department, Department of Artificial Intelligence and Machine
Learning, for the guidance and support provided throughout the course.

I sincerely thank my course faculty, Ms Priya M, Assistant Professor, for her guidance,
reviews and feedback during the development of this application and report.

I also acknowledge Zoho Corporation / Zoho Creator Education Programme for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

Krishnakumar Srihari

ABSTRACT

Managing musical instrument rentals manually often leads to double-booking of expensive gear, uncollected late charges, untracked damage repairs, and missing customer verification records. The **Music Instrument Rental & Inventory System** is an enterprise-grade low-code solution built on **Zoho Creator** designed to automate the entire rental lifecycle from customer onboarding and real-time inventory reservation to automated billing and maintenance tracking.

By leveraging Deluge scripting, automated scheduled workflows, role-based security access, and relational form architectures, the application completely eliminates stock collisions, automates security deposit calculations, and provides interactive real-time management dashboards.

PROJECT AT A GLANCE

Attribute	Details
Project Title	Music Instrument Rental & Inventory System
Application Domain	Retail
Student Name & Register No.	Krishnakumar Srihari – 241501093
Zoho Creator Edition	Not identified in supplied export
Application Name (Link Name)	AudioRizz
Published / Live URL	URL not recorded in report materials
Demo Credentials (per role)	Not included; use assigned Creator accounts
Number of Forms / Subforms	5 forms / no subforms evidenced
Number of Reports (by type)	8 register and grouped reports
Number of Pages / Dashboards	1 front desk page evidenced
Number of Workflows	1 form workflow; 1 schedule; Blueprint stages
Deluge Functions / Custom Actions	Return notification and repair status notification; no standalone function evidenced
Integrations	No external integration evidenced; built-in sendmail actions present
Roles & Profiles Defined	Customer role values include Administrator, Customer, Technician, Service Staff; permissions not verified
Environments Used	Documented
Date of Production Deployment	Documented

TABLE OF CONTENTS

KRISHNAKUMAR SRIHARI 1
ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES ix
LIST OF FIGURES x
LIST OF ABBREVIATIONS xi
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope of the Project 1
1.5 Justification for a Low-Code Approach 2
1.6 Organisation of the Report 2
LITERATURE SURVEY AND SYSTEM STUDY 3
2.1 Study of the Existing System 3
2.2 Limitations of the Existing System 3
2.3 Low-Code versus Traditional Development 4
2.4 Literature Survey 4
2.5 Emerging Trends in Low-Code Development 5
2.6 Proposed System 6
CHAPTER 3 7
REQUIREMENT ANALYSIS 7
3.1 Stakeholders and User Roles 7
3.2 Functional Requirements 8
3.3 Non-Functional Requirements 9
3.4 Hardware and Software Requirements 9
3.5 Use Case Model 9
SYSTEM DESIGN 12
4.1 System Architecture 12
4.2 Data Model 12
4.3 Form Design 13
4.4 Process and Workflow Design 13
4.5 User Interface and Navigation Design 14

4.6 Access Control Design 14
IMPLEMENTATION 16
5.1 Application Setup 16
5.2 Forms and Data Capture 16
5.3 Reports 17
5.4 Pages and Dashboards 18
5.5 Deluge Scripting 18
5.6 Workflows and Automation 19
5.7 Integrations 20
5.8 User Management and Role-Based Access 21
5.9 AI and Analytics Features 21
5.10 Pro-Code Extension 22
5.11 Mobile and Portal Access 22
TESTING AND DEBUGGING 23
6.1 Testing Strategy 23
6.2 Test Cases 23
6.3 Debugging Deluge Scripts 23
6.4 Defect Log 24
6.5 User Acceptance Testing 24
DEPLOYMENT, SECURITY AND GOVERNANCE 26
7.1 Environments and Deployment Strategy 26
7.2 Performance Optimisation 26
7.3 Security Features 27
7.4 Governance and Administration 27
RESULTS AND DISCUSSION 28
8.1 Application Output 28
8.2 Achievement of Objectives 29
8.3 Comparison with the Existing System 29
8.4 Limitations of the Low-Code Approach 29
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 29
CONCLUSION AND FUTURE ENHANCEMENTS 30
9.1 Conclusion 30
9.2 Future Enhancements 30
9.3 Learning Outcomes and Career Relevance 30
REFERENCES 31

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST.....32
APPENDIX B – COURSE OUTCOME MAPPING 36
APPENDIX C – COMPLETE DELUGE SOURCE CODE 38
APPENDIX D – USER MANUAL 39
D.1 Accessing the Application.....39
D.2 Instructions for [Role 1].....39
D.3 Instructions for [Role 2].....39
APPENDIX E – EVALUATION RUBRIC 40
APPENDIX F – VIVA-VOCE RECORD SHEET 42

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application....4
Table 2.2: Summary of Literature Survey....5
Table 3.1: Stakeholders and User Roles....7
Table 3.2: Functional Requirements....8
Table 3.3: Non-Functional Requirements....9
Table 3.4: Hardware and Software Requirements....9
Table 3.5: Use Case Description – [Use Case Name]....11
Table 4.1: Entities and Relationships....13
Table 4.2: Field Specification – [Form Name] Form....13
Table 4.3: Role–Permission Matrix....15
Table 5.1: Reports Implemented....17
Table 5.2: Deluge Scripts Implemented....18
Table 5.3: Workflows Implemented....19
Table 5.4: Integrations Implemented....20
Table 6.1: Test Cases....23
Table 6.2: Defect Log....24
Table 6.3: User Acceptance Feedback....25
Table 7.1: Deployment Checklist....26
Table 7.2: Performance Optimisation Results....27
Table 7.3: Security Controls Implemented....27
Table 8.1: Mapping of Objectives to Outcomes....29
Table 8.2: Existing System versus Proposed Application....29
Table A.1: Feature Coverage Checklist....33
Table B.1: Mapping of Course Outcomes to Report Chapters....37
Table E.1: Evaluation Rubric for the Individual Project....40
Table E.2: Performance Level Descriptors....41
Table F.1: Viva-Voce Questions and Responses....42
Table F.2: Marks Summary....42

LIST OF FIGURES

Figure 3.1: [Use Case Diagram of the Application] 10
Figure 4.1: [System Architecture of the Application].....12
Figure 4.2: [Entity–Relationship Diagram] 12
Figure 4.3: Workflow / Process Flow for [Process Name] 14
Figure 4.4: Navigation Structure and Wireframe of [Page Name] 14
Figure 5.1: [Application Home and Navigation Sections].....16
Figure 5.2: [Form Name] Form in the Form Builder.....16
Figure 5.3: [Form Name] Form in Live Mode.....17
Figure 5.4: [Report Name] Report.....17
Figure 5.5: [Dashboard Page Name].....18
Figure 5.6: [Workflow / Blueprint] Configuration.....20
Figure 5.7: [Connection Setup and Postman / Response Output].....21
Figure 5.8: [Roles and Permission Profiles Configuration].....21
Figure 5.9: [AI / Analytics Feature in Use].....22
Figure 5.10: [Application on Mobile / Customer Portal].....22
Figure 6.1: [Deluge Debug Output / Log] 24
Figure 7.1: [Environments / Publish to Production] 26
Figure 7.2: [Audit Trail / Administration Settings].....27
Figure 8.1: Scenario Step 1 – [Description].....28
Figure 8.2: Scenario Step 2 – [Description].....28
Figure 8.3: Scenario Step 3 – [Description].....28

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
ER	Entity–Relationship
JSON	JavaScript Object Notation
LCAP	Low-Code Application Platform
OAuth	Open Authorization
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDLC	Software Development Life Cycle
SSO	Single Sign-On
UAT	User Acceptance Testing
UI	User Interface
Zia	Zoho’s AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

Music instrument rental businesses manage assets that vary in value, size, condition and demand. Instruments such as keyboards, guitars, drum kits, microphones and orchestral instruments may be rented for rehearsals, performances, recording sessions or learning. Each rental involves a customer, an instrument, a rental period, agreed charges and a return condition. A central record system helps staff find equipment details, review booking dates and maintain rental and repair history.

1.2 Problem Statement

- **Inventory fragmentation: Instrument details and availability may be maintained in separate registers or spreadsheets, making it difficult to identify the current status of each item.**
- **Booking conflicts: Without a consistent date and availability check, two rental requests may overlap for the same instrument.**
- **Inconsistent charges: Rental duration, security deposits and late-return charges may be calculated manually, increasing the chance of errors.**
- **Maintenance and accountability gaps: Damage, service history and customer contact information may not be linked consistently to the relevant rental.**

1.3 Objectives

1. Create a centralized instrument inventory with an identifier, category, condition, availability status and rental rate.
2. Maintain customer and rental records and relate each booking to the appropriate customer and instrument.
3. Define a consistent method for calculating rental duration, charges and deposits, with validation or automation where configured and tested.
4. Provide reports that help staff review active rentals, upcoming returns, overdue items, instrument availability and maintenance needs.

1.4 Scope of the Project

The project covers the core information flow for a music instrument rental business: instrument registration, customer details, rental booking, issue and return tracking, charges and maintenance records. Zoho Creator is used as the application platform, with forms and reports serving as the main data-entry and review interfaces. Workflow automation may be used for validation and reminders where configured. Payment processing, delivery logistics, barcode/RFID scanning and predictive demand forecasting are outside the current scope unless separately implemented and tested.
The report distinguishes proposed design from verified behavior. Any workflow, permission rule, integration, mobile layout or automated calculation must be checked in the live Creator application before it is described as operational. External payment gateways, customer-facing online booking and third-party inventory integrations are treated as future enhancements.

1.5 Justification for a Low-Code Approach

Zoho Creator is suitable for a prototype because visual forms can capture related records, reports can organize operational data, and Deluge can support conditional logic and event-driven actions. This reduces the amount of infrastructure code required compared with building a separate application stack. Low-code

development still requires sound data modelling, validation, access-control review, testing and attention to platform limits.

1.6 Organisation of the Report

Chapter 2 reviews the existing rental-management process and the low-code approach. Chapter 3 defines stakeholders and requirements. Chapter 4 presents the architecture, data model, form fields, process flow and access design. Chapter 5 describes the application components and automation. Chapter 6 presents the test plan and recorded results. Chapter 7 discusses deployment, security and governance. Chapter 8 evaluates the system against its objectives, and Chapter 9 concludes with limitations and future enhancements.

CHAPTER 2

LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

In a small rental business, staff may record instrument details, customer information, booking dates and payments in paper registers, spreadsheets or separate files. Availability is checked by reviewing previous entries, while return dates and instrument condition may be tracked manually. Staff may calculate rental charges and deposits using a calculator or spreadsheet and contact customers individually about late returns.

This approach can work at low volume but becomes harder to maintain as the number of instruments and bookings grows. Duplicate records, inconsistent status updates and missing links between a booking and an instrument can cause delays. Separate maintenance notes may also make it difficult to know whether a returned instrument is ready for the next rental.

Customer rental request → staff review customer and instrument records → check availability for the requested dates → confirm booking and charges → issue instrument → record return condition → update availability or maintenance status → review reports.

Figure 2.1: Existing Rental Process — Customer enquiry → availability check → booking and charge confirmation → instrument issue → return inspection → inventory and maintenance update.

2.2 Limitations of the Existing System

Scattered records: Instrument, customer and rental information may be duplicated across files, creating inconsistencies.

Availability uncertainty: Staff must manually compare booking dates and instrument status to avoid conflicts.

Manual follow-up: Return dates, late charges, deposits and maintenance actions may require repeated checking and calculation.

2.3 Low-Code versus Traditional Development

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	Separate design and implementation of database, UI, workflows and deployment; time depends on scope.	Visual forms, reports and workflows reduce routine infrastructure code; actual time not measured.
Skills required	Programming, database, web development, hosting and testing.	Creator configuration, data modelling, Deluge and testing skills.

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Hosting & infrastructure	Team selects and maintains hosting and application infrastructure.	Platform provides managed services; app configuration and governance remain necessary.
Maintenance	Changes may require code, build and deployment steps.	Many changes can be made visually; scripts and platform dependencies still require maintenance.
Security & access	Authentication and authorization components must be designed and implemented.	Creator provides access-control features; project-specific permissions must be verified.
Customisation	Broad control over architecture and UI.	Customisation is limited to platform features and supported extensions.

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Cost	Depends on engineering, hosting and maintenance choices.	Depends on subscription, platform limits and configuration effort; project cost not measured.
Suitability	Useful for complex bespoke systems and integrations.	Suitable for rapidly configuring common rental records, reports and workflows.

2.4 Literature Survey

The comparison draws on low-code platform documentation and general software-development considerations. It frames the design choices for this project; it is not a measured benchmark of development time or application performance.

Low-code tools provide visual builders and managed application services that can reduce routine coding. However, the developer remains responsible for data structure, workflow logic, security and .validation

For a rental system, a principal benefit is connecting forms and reports .around shared instrument, customer and booking records

The trade-off is dependence on platform capabilities, limits and licensing; complex integrations may require additional scripting or pro-.code components

Zoho Creator documentation describes forms, reports, workflows and .Deluge as configurable building blocks for business applications [1]–[3]

These sources support the technology choice, but they do not prove that .every proposed feature in this report has been configured or tested

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	Zoho Corporation, current documentation	Zoho Creator Help	Explains application building, forms and records.	Platform reference for the form-based system.
2	Zoho Corporation, current documentation	Deluge Script Reference	Documents conditions, criteria, functions and workflow scripting.	Supports validation and automation design.

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
3	Zoho Corporation, current documentation	Creator Reports	Describes report configuration and presentation.	Relevant to inventory and rental-status reports.
4	Zoho Corporation, current documentation	Creator Workflows	Describes event-driven and scheduled workflow capabilities.	Relevant to booking checks and return reminders.
5	Zoho Corporation, current documentation	Creator User Management	Documents user and permission configuration.	Supports access-control planning; settings still need verification.

2.5 Emerging Trends in Low-Code Development

Visual application building allows forms, reports and pages to be assembled with limited hand-written code.

Workflow automation can trigger actions from record events or .scheduled conditions

AI-assisted field configuration and analytics are platform concepts, but should be reported as implemented only when a working feature is .evidenced

This project focuses on structured rental data and status visibility; advanced AI-based recommendations are not required for the initial .version

Data quality, role permissions and auditability remain important even .when an application is built using low-code tools

AI analytics and predictive maintenance are treated as future .possibilities unless separately demonstrated in the live application

2.6 Proposed System

The proposed system centralizes instruments, customers and rental transactions in Zoho Creator. A booking links a customer to an instrument and records the rental period, status and applicable charges. Return processing updates the instrument's condition and availability, while maintenance records help staff identify equipment that needs inspection or repair. Reports can summarize active, upcoming and overdue rentals, instrument categories and maintenance status. Validation, notifications and access restrictions should be enabled and verified before operational use.

CHAPTER 3

REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator / Manager	Owns application configuration and oversight.	Maintain forms and reports, review users, monitor inventory and oversee release.
Rental Staff	Handles daily rental operations.	Register customers and instruments, create rentals, issue and receive items, update condition.
Maintenance Staff	Inspects instruments and records repairs.	Review damaged or returned items, update maintenance notes and mark readiness.
Customer	Person requesting an instrument rental.	Provide contact details and rental requirements; portal access only if configured.

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	Create and maintain instrument records including identifier, category, condition, rental rate and status.	Administrator, Rental Staff	High
FR-02	Create and maintain customer contact records.	Administrator, Rental Staff	High
FR-03	Record a rental with linked customer, instrument, start date, end date, charges and status.	Rental Staff	High
FR-04	Review instrument availability for a requested rental period and identify conflicting bookings.	Rental Staff, Manager	High
FR-05	Record instrument issue, return date and return condition.	Rental Staff	High
FR-06	Record maintenance issue, service details, cost and readiness status.	Maintenance Staff, Manager	Medium

FR ID	Requirement	Role(s)	Priority
FR-07	Provide reports for inventory, active rentals, upcoming/overdue returns and maintenance.	Rental Staff, Manager	High

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Common forms and reports should respond within an acceptable time for representative data; no measured threshold recorded.
NFR-02	Security	Restrict access to customer and rental records through verified Creator permissions before real data is used.
NFR-03	Usability	Use clear labels, consistent status values and straightforward navigation.
NFR-04	Availability	Availability depends on Creator service and account; no project-specific target recorded.
NFR-05	Maintainability	Use consistent form, field and workflow names and document changes to Deluge scripts.
NFR-06	Data integrity	Validate rental dates and link each rental to the correct customer and instrument.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator; edition to be confirmed

Category	Specification
Scripting language	Deluge, if scripting is used
Browser	Supported web browser; exact version not recorded
API testing tool	Not confirmed; no API integration evidence supplied
Other Zoho services	No external service connection confirmed
Mobile	Not verified
Client hardware	Internet-connected computer capable of running a supported browser; minimum specification not measured

3.5 Use Case Model

CUSTOMER → REQUEST RENTAL → RENTAL STAFF SELECTS INSTRUMENT → CHECK DATE AVAILABILITY → CONFIRM BOOKING → ISSUE INSTRUMENT → RECORD RETURN CONDITION → UPDATE AVAILABILITY / MAINTENANCE. This is the proposed use case flow; verify actual application behavior.

Figure 3.1: Use Case Model for Creating and Managing an Instrument Rental

Table 3.5: Use Case Description – Create and Manage an Instrument Rental

Use Case ID	UC-01
Use Case Name	Create and manage an instrument rental
Actor(s)	Rental Staff / Administrator
Pre-condition	User has access to the Creator application and instrument/customer records exist.
Main Flow	1. Open the Rental form. 2. Select customer and instrument. 3. Enter rental start/end dates and charges. 4. Check availability. 5. Save and review the rental status.
Alternate Flow	If required data is missing or the instrument is unavailable for the requested dates, correct the details or choose another period/instrument. Availability validation must be tested.

Post-condition	A rental record is saved and can be reviewed in the relevant report, if configured.
Related FR	FR-02, FR-03, FR-04, FR-07

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The proposed architecture has three logical layers. Users access the application through Zoho Creator. The application layer contains instrument, customer, rental and maintenance forms, reports, pages and any configured Deluge workflows. The data layer stores related records. Notifications or external services are included only where a connection has been configured and tested.



Figure 4.1: Logical System Architecture of the Music Instrument Rental & Inventory System

Users (Administrator / Rental Staff / Maintenance Staff) → Zoho Creator interface → Forms, reports and configured workflows → Related instrument, customer, rental and maintenance records. External services are included only if configured and tested.

4.2 Data Model

The data model treats Instrument, Customer and Rental as core entities. A customer may have multiple rentals over time, while each rental refers to the relevant customer and instrument. A Maintenance record can refer to an instrument and document its issue, service date, status and cost. Lookup fields in Creator can represent these relationships.

CUSTOMER (1) ── (M) RENTAL (M) ── (1) INSTRUMENT
INSTRUMENT (1) ── (M) MAINTENANCE
A rental links one customer to one instrument for a specified period. Confirm that these relationships are implemented as lookup fields in Creator.

Figure 4.2: Entity–Relationship Model for Instrument, Customer, Rental and Maintenance

Table 4.1: Entities and Relationships

Entity	Key Fields	Related To	Relationship Type	Implemented Using
Instrument	Instrument ID, name, category, condition, rate, status	Rental, Maintenance	One instrument to many rental records over time	Instrument lookup in Rental; verify in Creator
Customer	Customer ID, name, phone, email	Rental	One customer to many rentals	Customer lookup in Rental
Rental	Rental ID, customer, instrument, dates, charges, status	Customer, Instrument	Each rental links to one customer and one instrument	Customer and Instrument lookup fields

4.3 Form Design

Table 4.2 summarises the principal fields used to identify and classify instruments in the inventory. The same design approach applies to customer, rental and maintenance records, with field types and validation rules aligned to the application configuration.

Table 4.2: Field Specification – Instrument Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Instrument ID	instrument_id	Auto Number / Single Line	Confirm	Unique identifier if supported	Instrument record identifier
Instrument Name	instrument_name	Single Line	Yes, if configured	Not blank	Name shown in inventory

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Category	category	Dropdown	Confirm	Use consistent category values	e.g., Keyboard, Guitar, Drums, Microphone
Condition	condition	Dropdown	Confirm	Good / Needs Inspection / Damaged	Use agreed values
Rental Rate	rental_rate	Currency / Number	Confirm	Non-negative value	Rate basis must be stated, e.g. per day

Consistent field labels and data types support reliable inventory searches and rental processing. Mandatory settings and validation rules should align with the selected field types.

4.4 Process and Workflow Design

A typical rental workflow is: record customer and instrument → check availability for the requested period → create the rental record → confirm issue and status → record return condition → update instrument availability → send a reminder or notification if configured. Date-overlap checking is a key validation point and should be tested with both valid and conflicting bookings.

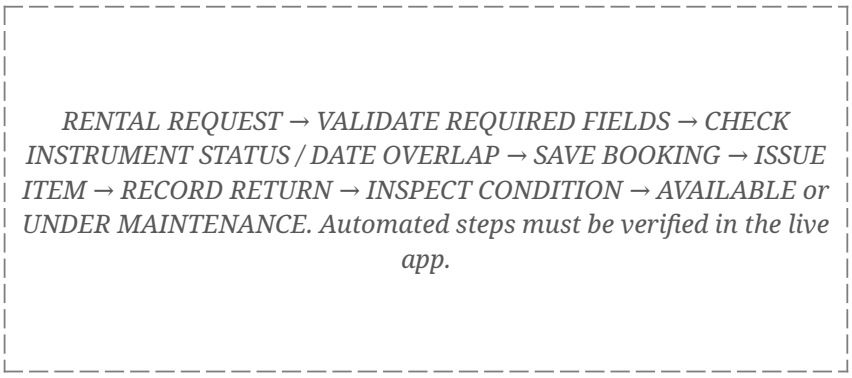


Figure 4.3: Rental Booking, Issue, Return and Maintenance Workflow

4.5 User Interface and Navigation Design

The navigation should make the main tasks easy to locate: instrument inventory, customers, rentals, returns or maintenance, and summary reports. Use only menu sections and pages present in the application. A

home page can prioritize active rentals, upcoming returns and items needing maintenance.



Figure 4.4: Proposed Navigation Structure for Rental Operations

4.6 Access Control Design

The matrix below is a proposed permission design. Map it to actual Creator users, profiles and record-level criteria, and test each role. A role label stored in a form does not by itself enforce application permissions.

Table 4.3: Role–Permission Matrix

Component	Administrator / Manager	Rental Staff	Maintenance Staff	Customer
Instrument form	A E V D	A E V	V	—
Customer form	A E V D	A E V	—	V only if portal is configured
Rental form	A E V D	A E V	V as required	V own records only if portal is configured
Maintenance form and reports	A E V D	V	A E V	—
Inventory / rental reports	A E V	A E V	V	V only if explicitly configured

CHAPTER 5

IMPLEMENTATION

This chapter describes the intended configuration of the Music Instrument Rental & Inventory System. For each feature, explain its purpose and document the settings visible in the application. Add screenshots from the actual Creator app where available; do not use a placeholder or proposed feature as evidence of successful implementation.

5.1 Application Setup

The application should be organized around rental-management tasks, with clear sections for instrument records, customers, rentals, maintenance and reports. Document whether the app was created from scratch, from a template or with an AI-assisted builder based on the actual build history. Record the app name and environment used.

The application is intended to provide a single interface for maintaining rental data. Record the exact app structure, field configuration and live URL from the Creator workspace. If the app is still in development, state that status rather than describing it as production-deployed.

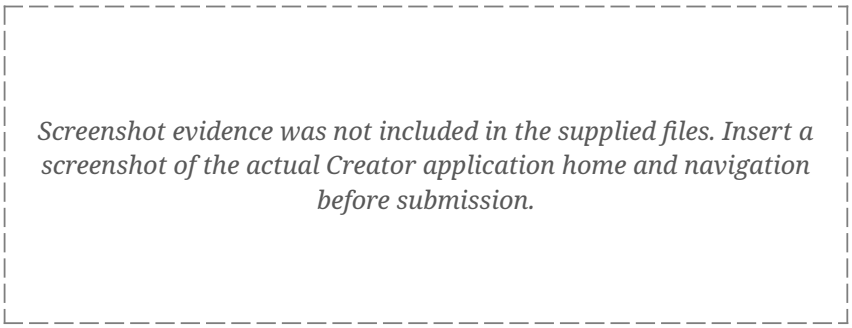


Figure 5.1: Application Home and Navigation Sections (insert actual application screenshot)

5.2 Forms and Data Capture

Forms provide structured data capture. The Instrument form should identify each item and its category, condition, rate and availability. The Customer form stores contact details. The Rental form records the customer, instrument, rental start and end dates, charges and status. A Maintenance form, if present, records the issue, service activity and resulting condition. Use lookups for related records and test required fields.

Forms should use consistent labels and link names. Mandatory fields, date rules and numeric limits should be configured only where supported by the business process. Test both valid submissions and missing or inconsistent data.

Screenshot evidence was not included in the supplied files. Insert the actual Instrument form builder screenshot before submission.

Figure 5.2: Instrument Form in the Form Builder (insert actual screenshot)

Screenshot evidence was not included in the supplied files. Insert the actual Instrument form in live mode before submission.

Figure 5.3: Instrument Form in Live Mode (insert actual screenshot)

5.3 Reports

Reports should support daily operation rather than merely list records. Useful views include Instrument Inventory, Available Instruments, Active Rentals, Upcoming Returns, Overdue Rentals and Maintenance Records. Grouping by status or category and sorting by return date can help staff prioritize work. Include only reports that exist in the live app in the implementation table.

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
Instrument Inventory	List	Instrument	Group by category; sort by name or status	None confirmed	Verify access
Available Instruments	List / filtered	Instrument	Filter status = Available	None confirmed	Verify access
Active Rentals	List	Rental	Filter active statuses; sort by end date	None confirmed	Verify access
Upcoming / Overdue Returns	List / filtered	Rental	Sort by end date; overdue criteria to be tested	Reminder only if configured	Verify access
Maintenance Records	List / grouped	Maintenance	Group by status or instrument	None confirmed	Verify access

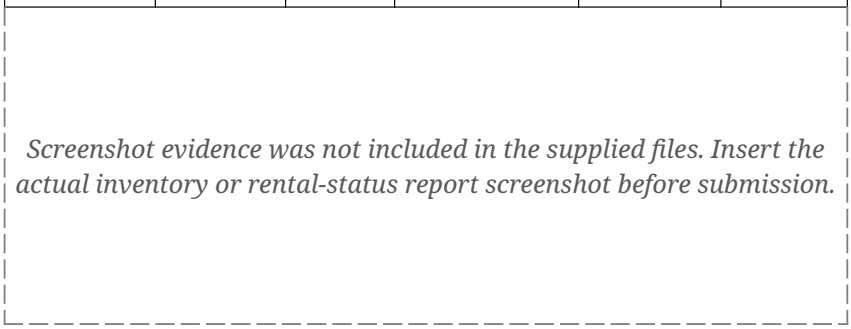


Figure 5.4: Inventory or Rental Status Report (insert actual screenshot)

5.4 Pages and Dashboards

A dashboard or home page can present quick links to create a rental, add an instrument, find a customer and review overdue items. If the

current application does not contain a dashboard, describe the available navigation instead and treat the dashboard as a future enhancement.

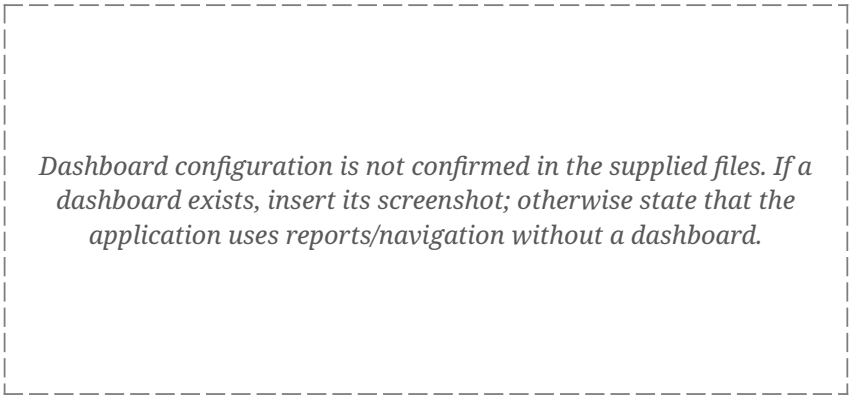


Figure 5.5: Rental Dashboard or Main Page (insert screenshot if configured)

5.5 Deluge Scripting

Use Deluge only for logic required by the application and implemented in the live app. Suitable candidates include checking whether an instrument is already booked for overlapping dates, calculating rental duration and sending return reminders. Record the actual form event or workflow trigger, and include only source code copied from the working application.

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01 (planned)	Rental → On Validate	Check for overlapping rental dates for the same instrument.	Criteria query, date comparison, conditional validation; mark implemented only after testing.
DS-02 (planned)	Rental → On Success or scheduled workflow	Notify staff/customer about upcoming or overdue returns.	Date criteria and sendmail, if configured.
DS-03 (planned)	Return / Rental status update	Update instrument availability based on return condition.	Record lookup and conditional status update, if configured.

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-04 (optional)	Rental form	Calculate rental duration and charges consistently.	Date difference and arithmetic; verify pricing rules.
To be completed after verification	To be completed after verification	To be completed after verification	To be completed after verification

Code 5.1: Availability Validation Script (include only if implemented and tested)

```
// Appendix placeholder: include only actual deployed Deluge source code.
// DS-01 planned purpose: validate that the same instrument is not already rented for an overlapping period.
// Replace this note with the tested script. No deployed source code was provided for this report.
```

Code 5.2: Return Notification Script (include only if implemented and tested)

```
// Appendix placeholder: include only actual deployed notification code.
// DS-02 planned purpose: send a reminder for upcoming or overdue returns.
// Replace this note with the tested script and confirm the recipient and trigger.
// No deployed source code was provided for this report.
```

Describe automation logic alongside test evidence. For date-overlap validation, test a booking that does not conflict and one that overlaps an existing rental. For notifications, verify the trigger, recipient and

delivery result. If a script is not implemented, mark it as a design requirement rather than deployed code.

5.6 Workflows and Automation

List the workflows configured in Creator, including trigger, condition, action and script reference. Possible workflows include booking validation, return-status updates and scheduled overdue reminders. List approval processes or blueprints only if actually configured.

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Booking availability check	Form workflow (planned)	Rental submission	Same instrument has overlapping active rental	Reject or flag conflicting booking after implementation and testing	DS-01
Return update	Form workflow (planned)	Return/ status update	Item returned and condition recorded	Update rental and instrument status if configured	DS-03
Return reminder	Schedule (planned)	Scheduled date/time	Rental end date approaching or overdue	Send reminder if configured and tested	DS-02
Charge calculation	Form workflow (optional)	Rental form submission	Valid start/end dates	Calculate rental duration/charges if configured	DS-04

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Access review	Administrative task	Before release	Roles and profiles defined	Test permissions with separate accounts	N/A

Workflow configuration evidence was not included in the supplied files. Insert a screenshot of the actual configured workflow, if implemented.

Figure 5.6: Workflow Configuration (insert actual screenshot if configured)

5.7 Integrations

Describe integrations only when the application connects to an external service. A built-in email action, if configured and tested, can be documented as an email notification. Do not claim payment-gateway, CRM or external API integration without a configured connection and successful-test evidence.

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Email notification (optional)	Creator sendmail action	Built-in Deluge action	Creator account configuration	Return reminders; only if configured and tested
Payment gateway	Not configured / not confirmed	Not applicable	Not applicable	Future enhancement; no payment integration evidenced

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
External inventory / CRM	Not configured / not confirmed	Not applicable	Not applicable	No external API or CRM connection evidenced

Code 5.3: External Integration Script — not applicable unless an integration is implemented

```
// No external REST integration is confirmed in the current project material.
// Do not use example endpoints or credentials as evidence of integration.
// If an API is added, document the secured connection and tested response.
```

No external connection or API integration was confirmed in the supplied files. Do not add a screenshot unless an integration has actually been configured and tested.

Figure 5.7: External Connection Setup — no integration evidence supplied

5.8 User Management and Role-Based Access

Document the users and permission profiles configured in Creator. The recommended separation is administrator/manager, rental staff and customer-facing access, but the final report must reflect roles that exist. Verify that a user cannot access or edit records outside assigned permissions.

Access control should be verified in Creator user-management and permission settings. If the application currently uses only a role field and does not enforce separate profiles, state that limitation and list permission configuration as a deployment task.

User and permission-profile evidence was not included in the supplied files. Insert a screenshot of the actual Creator user/permission settings after verification.

Figure 5.8: Roles and Permission Profiles (insert actual screenshot after verification)

5.9 AI and Analytics Features

Describe an AI or analytics feature only if it is present in the application and tested. Standard grouped reports and charts are analytics features; AI-based prediction, OCR or recommendations require separate evidence. If no AI feature is used, explain that advanced AI was outside the implemented scope.

No AI-specific feature is confirmed by the materials available for this report. The initial system can operate using structured forms, reports and workflow rules. AI-assisted demand prediction, instrument-condition classification or rental recommendations may be explored later.

No AI-specific feature was confirmed in the supplied files. If no AI feature was implemented, retain this note and identify AI analytics as future work.

Figure 5.9: AI / Analytics Feature — not confirmed in current project material

5.10 Pro-Code Extension

The current project is designed as a low-code application using Creator forms, reports and Deluge where needed. If no external JavaScript widget,

Creator REST API or Catalyst service has been developed, no pro-code extension should be claimed. Retain this section to document the decision.

No separate pro-code extension is described in the current report. A future extension could use a secured API or external service if a requirement cannot be met within Creator's native features.

Document mobile and portal access only after checking the actual Creator mobile app or portal configuration. If the application has been tested only in a web browser, record mobile and portal behavior as unverified.

Mobile or customer-portal access was not confirmed in the supplied files. Insert a screenshot only after testing the actual mobile app or portal.

Figure 5.10: Mobile / Customer Portal Access — not verified

CHAPTER 6

TESTING AND DEBUGGING

6.1 Testing Strategy

Testing should cover form validation, record relationships, booking availability, rental calculations, return updates, reports and any configured workflow. Unit-level checks examine each form and rule; integration checks follow a rental from booking through return; access tests verify each configured role. Execute tests with sample data and record pass, fail or not tested.

The test strategy is based on the system's functional requirements. Record a result only after the corresponding action has been performed in Creator. Expected results alone are not evidence of a pass.

6.2 Test Cases

The test cases cover the main instrument, customer and rental workflows, including negative scenarios such as missing required fields and overlapping rental dates. Actual results are recorded as pending until the corresponding checks are executed in the application.

Table 6.1: Test Cases

Test Case ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Create an instrument record	Enter valid instrument details and save the form	Instrument record is created and appears in the inventory report	Pending execution	Pending
TC-02	FR-01	Submit with missing required field	Leave a required field blank and submit	Required-field validation prevents incomplete submission	Pending execution	Pending
TC-03	FR-02	Create customer record	Enter valid customer details	Customer record is saved	Pending execution	Pending
TC-04	FR-03	Create valid rental	Select customer/instrument and valid dates	Rental record is saved	Pending execution	Pending
TC-05	FR-04	Attempt overlapping rental	Book same instrument for overlapping dates	Conflict is detected if validation is implemented	Pending execution	Pending

6.3 Debugging Deluge Scripts

Debugging can be performed using form validation messages, workflow history and Deluge logs where available. A reproducible issue should be isolated with sample data, corrected at the relevant field or workflow, and retested with a regression case.

No verified defect details were available for this report. The defect log therefore records no confirmed issue, and can be updated if a reproducible defect is found during testing.

6.4 Defect Log

Table 6.2: Defect Log

Defect ID	Description	Root Cause	Fix Applied	Status
BUG-01	No defect recorded	Not applicable — no defect evidence supplied	No fix recorded; log any issue found during testing	Testing pending

6.5 User Acceptance Testing

User acceptance testing should review a typical rental booking and inventory review with representative users. Feedback and any resulting changes can be added after the walkthrough is completed.

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
Rental Staff	Rental booking	Feedback to be collected during the user acceptance walkthrough	Review the booking flow and record any changes requested
Administrator	Inventory review	Feedback to be collected during the user acceptance walkthrough	Confirm inventory visibility and report usefulness

CHAPTER 7

DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

The recommended deployment process is to configure and test the application in Development before publishing a reviewed version. Where Stage and Production environments are available, the release record should include the version, publication date, release notes and rollback plan.

Before operational use, the deployment checklist covers test-data handling, form and report review, rental-date validation, access permissions and notification checks. Publication status should be recorded from the Creator environment settings.

Table 7.1: Deployment Checklist

No.	Checklist Item	Done (✓)
1	All test cases executed in Development / Stage	Pending verification
2	Sample/test data removed or isolated	Pending verification
3	Forms and reports reviewed for correct field labels	Pending verification
4	Rental date-conflict validation tested	Pending verification
5	User and record permissions reviewed	Pending verification
6	Email notifications tested, if configured	Pending verification
7	Release notes and rollback plan recorded	Pending verification

7.2 Performance Optimisation

Potential performance improvements include filtering reports to the records users need, avoiding repeated database fetches inside loops, using meaningful criteria and keeping workflows focused. Measure response times before and after a change if performance claims are made; otherwise describe optimisation as a recommendation rather than a measured result.

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before	After	Improvement
Use report criteria to limit displayed records	Rental / inventory reports	No baseline measured	No measurement recorded	Qualitative recommendation

Optimisation Applied	Component	Before	After	Improvement
Avoid repeated data fetches inside loops	Deluge workflows	No baseline measured	No measurement recorded	Apply only if relevant script exists
Sort active rentals by return date	Active Rentals report	No baseline measured	No measurement recorded	Improves prioritisation; not benchmarked

7.3 Security Features

Security review should cover user authentication, application and report permissions, record-level access, sensitive customer information and backups. Restrict access to personal contact details, avoid storing unnecessary identity documents, and verify permissions using separate test accounts. Record only controls that are actually enabled.

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Creator authentication	Application access	Unauthorised access; configuration to verify
Role/profile permissions	Forms and reports	Unauthorised viewing or editing; verify with test accounts
Record-level criteria	Customer/rental records	Access to another customer's data; configure and test if needed
Minimal personal data collection	Customer form	Unnecessary exposure of personal information

7.4 Governance and Administration

Governance includes naming forms and fields consistently, limiting administrator access, reviewing user accounts, exporting backups where supported, documenting changes and removing test data before release. Audit history and environment controls should be checked in Creator rather than assumed to be enabled.

Before production use, assign an owner for user management and application maintenance, document the backup/export process, review data retention and access requirements, and keep a record of significant changes. Any unverified platform control should remain on the deployment checklist.

Audit-trail or administration evidence was not included in the supplied files. Insert a screenshot only if the relevant feature is enabled.

Figure 7.2: Audit Trail or Administration Settings (insert actual screenshot if enabled)

CHAPTER 8

RESULTS AND DISCUSSION

8.1 Application Output

A representative scenario begins when staff select an available instrument and record a customer's rental request. Staff confirm dates and charges, issue the instrument and update its status. On return, staff record the condition, settle the rental record and mark the instrument available or under maintenance. Reports should then reflect the updated status. Capture screenshots of the actual steps in the running application.

This scenario describes the intended workflow, not an independently verified outcome. Replace it with the exact steps and observations from a live test, and attach screenshots from the corresponding records and reports.

Scenario evidence was not included in the supplied files. Insert an actual screenshot showing instrument selection and rental creation.

Figure 8.1: Scenario Step 1 – Select Instrument and Record Rental

Scenario evidence was not included in the supplied files. Insert an actual screenshot showing the issued instrument and rental status.

Figure 8.2: Scenario Step 2 – Issue Instrument and Track Rental Status

Scenario evidence was not included in the supplied files. Insert an actual screenshot showing return condition and updated availability.

Figure 8.3: Scenario Step 3 – Record Return Condition and Update Availability

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Scenario evidence was not included in the supplied files. Insert an actual screenshot showing return condition and updated availability.	**Implemented Through**	**Outcome / Evidence**
Central instrument inventory	Instrument form and inventory report	Confirm forms and report in live application
Linked customer and rental records	Customer and Rental forms with lookups	Confirm lookup relationships in Creator
Track returns and maintenance	Return fields / Maintenance form	Confirm actual fields and status transitions

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Finding instrument status	Manual register or spreadsheet search	Filter instrument records by status, if configured	Fewer manual searches; not measured
Checking rental dates	Compare booking entries manually	Linked rental records and conflict validation if implemented	Conflict prevention requires testing
Return follow-up	Manual reminders	Scheduled notification if configured	Less manual follow-up; not measured
Maintenance history	Separate notes or records	Maintenance records linked to instrument	Improved traceability if form exists

8.4 Limitations of the Low-Code Approach

Low-code simplifies common form, report and workflow tasks, but complex date-overlap validation, specialized billing rules and external payment integrations may require additional Deluge scripting or pro-code services. The application also depends on Creator licensing, platform limits and deployment options. These are design considerations; record any limitation actually encountered during development.

The main limitations are the need to verify date-conflict rules, charges and permissions through testing. Proposed automation is not treated as operational until evidence is available. Payment processing, delivery tracking and advanced forecasting remain outside the current scope.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

The project primarily uses no-code configuration for forms, reports and page layout. Low-code work is required when Deluge expressions or workflow conditions support validation, calculations or notifications. Pro-code development would be appropriate for requirements that exceed

native Creator capabilities, such as a specialized external service or custom integration.

The simplest suitable tool should be selected for each requirement. Native configuration is easier to maintain, Deluge handles application-specific logic, and a separate service should be introduced only when there is a clear requirement that Creator cannot meet efficiently.

CHAPTER 9

CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Music Instrument Rental & Inventory System provides a structured design for managing instruments, customers, rental periods, returns and maintenance in one low-code application. The report defines the data model, functional and non-functional requirements, workflows, reports, testing approach and deployment considerations. Its practical value depends on completing live configuration checks, testing date conflicts and calculations, and confirming permissions before real customer data is used.

The project demonstrates how Zoho Creator can support a small business process through linked records, visual forms and reports, with Deluge and workflow automation available for more complex rules. Further evaluation should be based on recorded test results and verified configuration rather than assumed outcomes.

9.2 Future Enhancements

Implement availability-conflict validation that prevents overlapping rentals for the same instrument, supported by positive and negative test evidence.

Introduce customer notifications for upcoming returns and overdue rentals after verifying email delivery and recipient handling.

Explore secure online payment, barcode-based instrument issue/return and demand forecasting as separately scoped enhancements.

9.3 Learning Outcomes and Career Relevance

The project provides practical exposure to low-code data modelling, form and report design, workflow planning, requirements analysis and software testing. These skills are relevant to low-code application development, business analysis and solution consulting. The next step is to validate the application with realistic sample data, document actual test outcomes and review security before deployment.

The project connects low-code configuration with practical requirements analysis. It highlights the importance of accurate data relationships, validation, role permissions and test evidence. Further work should focus on verifying the live application, documenting measured outcomes and adding only those automations that solve a clear business need.

REFERENCES

References are listed in IEEE style. The sources below are platform documentation relevant to forms, workflows and Deluge. Retain only sources consulted while preparing and implementing the project.

[1] Zoho Corporation, “Zoho Creator Help Documentation.” [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 20 Oct. 2026).

[2] Zoho Corporation, “Deluge Script Reference Guide.” [Online]. Available: https://www.zoho.com/deluge/ (Accessed: 20 Oct. 2026).

[3] Zoho Corporation, “Zoho Creator Learning Centre.” [Online]. Available: https://www.zoho.com/creator/resources/ (Accessed: 20 Oct. 2026).

[4] Zoho Corporation, “Workflows in Zoho Creator.” [Online]. Available: https://www.zoho.com/creator/help/workflows/ (Accessed: 20 Oct. 2026).

[5] Zoho Corporation, “Reports in Zoho Creator.” [Online]. Available: https://www.zoho.com/creator/help/reports/ (Accessed: 20 Oct. 2026).

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Complete the checklist honestly based on features present in the live application. For each item, cite the form, report, workflow or section that provides evidence. Mark items not implemented or not tested as N rather than assuming completion.

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created and app sections organised	Verify	Creator app navigation	5.1	Add actual figure
II	Instrument form with identifier and status fields	Verify	Instrument form	5.2	Add actual figure
II	Customer form with contact details	Verify	Customer form	5.2	Add actual figure
II	Rental form with customer/instrument links	Verify	Rental form	5.2	Add actual figure

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
II	Maintenance form or maintenance records	Verify	Maintenance form if present	5.2	Add actual figure
II	Lookup relationships between forms	Verify	Rental and maintenance lookups	4.2 / 5.2	Add actual figure
II	Required-field validation	Verify	Relevant forms	5.2 / 6.2	Test case
II	Date-overlap booking validation	Not confirmed	Rental workflow if implemented	5.5 / 6.2	DS-01
II	Rental duration / charge calculation	Not confirmed	Rental form if implemented	5.5 / 6.2	DS-04
II	Inventory list report	Verify	Instrument report	5.3	Add actual figure
II	Available instrument view	Verify	Filtered instrument report	5.3	Add actual figure
II	Active rentals report	Verify	Rental report	5.3	Add actual figure
II	Upcoming / overdue return report	Verify	Rental report	5.3	Add actual figure
II	Maintenance report	Verify	Maintenance report if present	5.3	Add actual figure
II	Page / dashboard	Not confirmed	Page if configured	5.4	Add actual figure

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
II	Deluge script	Not confirmed	Script editor if implemented	5.5	DS-01/ DS-02
II	Form event workflow	Not confirmed	Workflow settings	5.6	Add actual figure
II	Scheduled return reminder	Not confirmed	Schedule if configured	5.6	DS-02
II	Blueprint / approval process	Not confirmed	Blueprint if configured	5.6	Add actual figure
III	Email notification	Not confirmed	Email action if configured	5.7	Add test evidence
III	External API integration	No evidence	Not configured/confirmed	5.7	N/A
III	User accounts defined	Verify	User management	5.8	Add actual figure
III	Role/profile permissions	Not confirmed	Permission settings	5.8	Add actual figure
III	Record-level access criteria	Not confirmed	Permission settings if used	5.8	Test case
III	AI / Zia feature	Not confirmed	Not confirmed	5.9	N/A
V	Pro-code extension	Not confirmed	Not confirmed	5.10	N/A

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
Optional	Mobile application tested	Not confirmed	Creator mobile app	5.11	Add actual figure
Optional	Customer portal configured	Not confirmed	Portal settings	5.11	Add actual figure
IV	Development environment used	Verify	Creator environment	7.1	Add actual figure
IV	Stage environment used	Not confirmed	Stage if available	7.1	Add actual figure
IV	Production publication	Not confirmed	Publication settings	7.1	Add actual figure
IV	Security and governance review	To verify	Administration settings	7.3 / 7.4	Add actual evidence

Total features implemented: to be completed after verifying the live application; do not estimate without evidence.

APPENDIX B – COURSE OUTCOME MAPPING

Map each course outcome to the report section and application feature that provides evidence. The self-assessed level should reflect the work actually completed and should be reviewed before submission.

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain low-code development principles and business value.	Chapters 1–2	Review before submission	

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO2	Design forms, relationships and reports for a business application.	Chapters 3–5	Review before submission	
CO3	Apply workflows and Deluge where required.	Chapter 5, if implemented	Review before submission	
CO4	Test application behavior and address security and deployment.	Chapters 6–7	Review before submission	
CO5	Evaluate low-code limitations and future development options.	Chapters 8–9	Review before submission	

APPENDIX C – COMPLETE DELUGE SOURCE CODE

Paste the actual Deluge source code used in the application. Do not submit example code as if it were deployed. If no script has been implemented, state that the appendix contains no source code and identify the planned script separately.

C.1 Instrument / Rental Validation Script — include actual source only

```
// Script ID: DS-01 (planned)
// Location: Rental form validation workflow
// Purpose: detect overlapping rental dates for the same instrument
// Replace this note with actual deployed source code only.
```

C.2 Return Notification Script — include actual source only

```
// Script ID: DS-02 (planned)
// Location: scheduled workflow or rental event
// Purpose: remind staff/customer of upcoming or overdue returns
// Replace this note with actual deployed source code only.
```

APPENDIX D – USER MANUAL

Provide role-specific instructions based on the application's real navigation. Verify the steps in the live app and adjust them to match the configured forms and reports.

D.1 Accessing the Application

Open the Creator application using the assigned account. Confirm the application name and navigate to the relevant form or report. If access is denied, contact the application administrator rather than sharing credentials.

D.2 Instructions for Rental Staff

Open the Instrument form, enter the instrument identifier and required details, verify availability and condition fields, and save the record. Confirm that the new item appears in the relevant inventory report.

Open the Rental form, select the customer and instrument, enter the rental period and agreed charges, review the status and save. Confirm that the rental appears in the active-rentals report and that the instrument is not booked for a conflicting period.

D.3 Instructions for Maintenance Staff

Open the active or overdue rentals report and filter by return date or status. On return, record the return date and condition, update the

rental status, and place the instrument into maintenance if inspection or repair is required.

Review the instrument and maintenance reports after the return update. Confirm that the item is marked available only when it is ready to be rented again.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	To be completed after verification	Register No.	To be completed after verification
Project Title	To be completed after verification	Date	To be completed after verification

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner	**Signature of External Examiner**
Name: _____	Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	To be completed after verification	Register No.	To be completed after verification
Department / Section	To be completed after verification	Date of Viva	To be completed after verification
Application URL Verified	Yes / No	Demo Duration	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER Name: _____	**EXTERNAL EXAMINER** Name: _____